

```

"""
StegSim receipts.py - Receipt and metrics emission.
Every evaluated transition produces a JSONL receipt.
Every step produces a metrics record.
Receipts are the replay and reconstruction surface.
"""

import json
import hashlib
from datetime import datetime, timezone
from pathlib import Path
from dataclasses import asdict
from .model import EvaluationResult, SystemState

def now() -> str:
    return datetime.now(timezone.utc).isoformat()

# — Receipt writer —————

class ReceiptWriter:
    """
    Appends JSONL receipts for every evaluated transition.
    Schema: vbds.receipt.v1
    """

    def __init__(self, path: str):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self._count = 0

    def write(self, result: EvaluationResult, run_id: str):
        receipt = {
            "receipt_version": "vbds.receipt.v1",
            "run_id": run_id,
            "receipt_index": self._count,
            "step": result.step,
            "agent_id": result.agent_id,
            "proposal_id": result.proposal_id,
            "local_policy_valid": result.local_policy_valid,
            "authority_valid_current_state": result.authority_valid_current_state,
            "viability_preserved": result.viability_preserved,
            "legitimacy_capacity_ok": result.legitimacy_capacity_ok,

```

```

        "viability_before":            round(result.viability_before, 4),
        "viability_after_projected":   round(result.viability_after_projecte
        "viability_delta":            round(result.viability_delta, 4),
        "authority_staleness":         round(result.authority_staleness, 4),
        "legitimacy_capacity":         round(result.legitimacy_capacity, 4),
        "autonomy_projected":         round(result.autonomy_projected, 4),
        "decision":                   result.decision,
        "reason":                     result.reason,
        "local_global_disagree":       result.local_global_disagree,
        "state_hash_before":           result.state_hash_before,
        "state_hash_after_projected":  result.state_hash_after_projected,
    }
    with open(self.path, "a") as f:
        f.write(json.dumps(receipt) + "\n")
    self._count += 1

def count(self) -> int:
    return self._count

# — Metrics writer —————

class MetricsWriter:
    """
    Appends JSONL metrics for every step (or every N steps).
    Schema: vbds.metrics.v1
    """

    DECISIONS = ("ALLOW", "DENY", "FLAG", "FAIL_CLOSED")

    def __init__(self, path: str):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self._step_results: list = []

    def accumulate(self, result: EvaluationResult):
        """Call once per evaluation result."""
        self._step_results.append(result)

    def flush_step(self, step: int, state: SystemState, run_id: str):
        """Write aggregated metrics for the current step, then reset buffer."""
        results = self._step_results
        n        = len(results) if results else 1

        counts = {d: sum(1 for r in results if r.decision == d) for d in self.DEC

        local_pass    = sum(1 for r in results if r.local_policy_valid)

```

```

global_allow = counts["ALLOW"]
lg_disagree  = sum(1 for r in results if r.local_global_disagree)

env = state.environment
v   = state.viability

metrics = {
    "schema":          "vbds.metrics.v1",
    "run_id":          run_id,
    "step":            step,
    "generated_at":    now(),

    # Viability
    "viability_composite": round(v.composite, 4),
    "viability_status":    v.status,
    "trust_continuity":    round(v.trust_continuity, 4),
    "authority_coherence": round(v.authority_coherence, 4),
    "resource_balance":    round(v.resource_balance, 4),
    "policy_freshness":    round(v.policy_freshness, 4),
    "mutation_pressure_comp": round(v.mutation_pressure, 4),
    "fragmentation_comp":  round(v.fragmentation, 4),

    # Environment
    "env_resource_scarcity": round(env.resource_scarcity, 4),
    "env_mutation_pressure": round(env.mutation_pressure, 4),
    "env_fragmentation":    round(env.fragmentation, 4),
    "env_policy_version":    env.policy_version,
    "env_policy_lag":        round(env.policy_lag, 4),
    "env_communication_fidelity": round(env.communication_fidelity, 4),
    "env_external_shock":    round(env.external_shock, 4),

    # Decision counts
    "total_proposals": n,
    "accepted_count": counts["ALLOW"],
    "denied_count":    counts["DENY"],
    "flagged_count":   counts["FLAG"],
    "fail_closed_count": counts["FAIL_CLOSED"],

    # Key derived metrics
    "local_policy_pass_rate": round(local_pass / n, 4),
    "global_admissibility_pass_rate": round(global_allow / n, 4),
    "local_global_disagreement_rate": round(lg_disagree / n, 4),
    "stale_authority_denial_rate": round(
        sum(1 for r in results if not r.authority_valid_current_state) /
    ),
}

```

```

        with open(self.path, "a") as f:
            f.write(json.dumps(metrics) + "\n")

        self._step_results = []
        return metrics

# — Phase diagram collector (v2+) —————

class PhaseDiagramCollector:
    """
    v2: Collects (trust_decay_rate, authority_staleness_rate, first_inadmissible_
    across multiple seeded runs to produce a phase diagram.
    Not active in v1 (enable_phase_diagram=False).
    """

    def __init__(self, path: str):
        self.path = Path(path)
        self.records = []

    def record(self, trust_decay: float, auth_staleness: float,
               first_warning: int, first_inadmissible: int, final_viability: float):
        self.records.append({
            "trust_decay_rate": trust_decay,
            "authority_staleness_rate": auth_staleness,
            "first_warning_step": first_warning,
            "first_inadmissible_step": first_inadmissible,
            "final_viability": final_viability,
        })

    def write(self):
        self.path.parent.mkdir(parents=True, exist_ok=True)
        with open(self.path, "w") as f:
            for r in self.records:
                f.write(json.dumps(r) + "\n")
        return len(self.records)

```